

# UNIFIED BILLING PLATFORM

## 1.INTRODUCTION

Modern enterprise transport operations involve multiple clients, vendors, and employees generating large amounts of trip data every day. Traditional manual billing leads to inconsistencies, delays, and errors.

This project presents a Unified Billing & Reporting Platform that automates trip processing, billing computation, incentive allocation, and monthly reporting using a **Python backend, MySQL database, and Streamlit frontend**.

The solution offers a scalable, multi-tenant architecture supporting clients, vendors, and employees with role-based authentication. It ensures accuracy, transparency, and auditability in billing workflows.

## 2. PROBLEM STATEMENT

MoveInSync partners with various corporate clients and transport vendors who use different billing models (package, trip-based, hybrid).

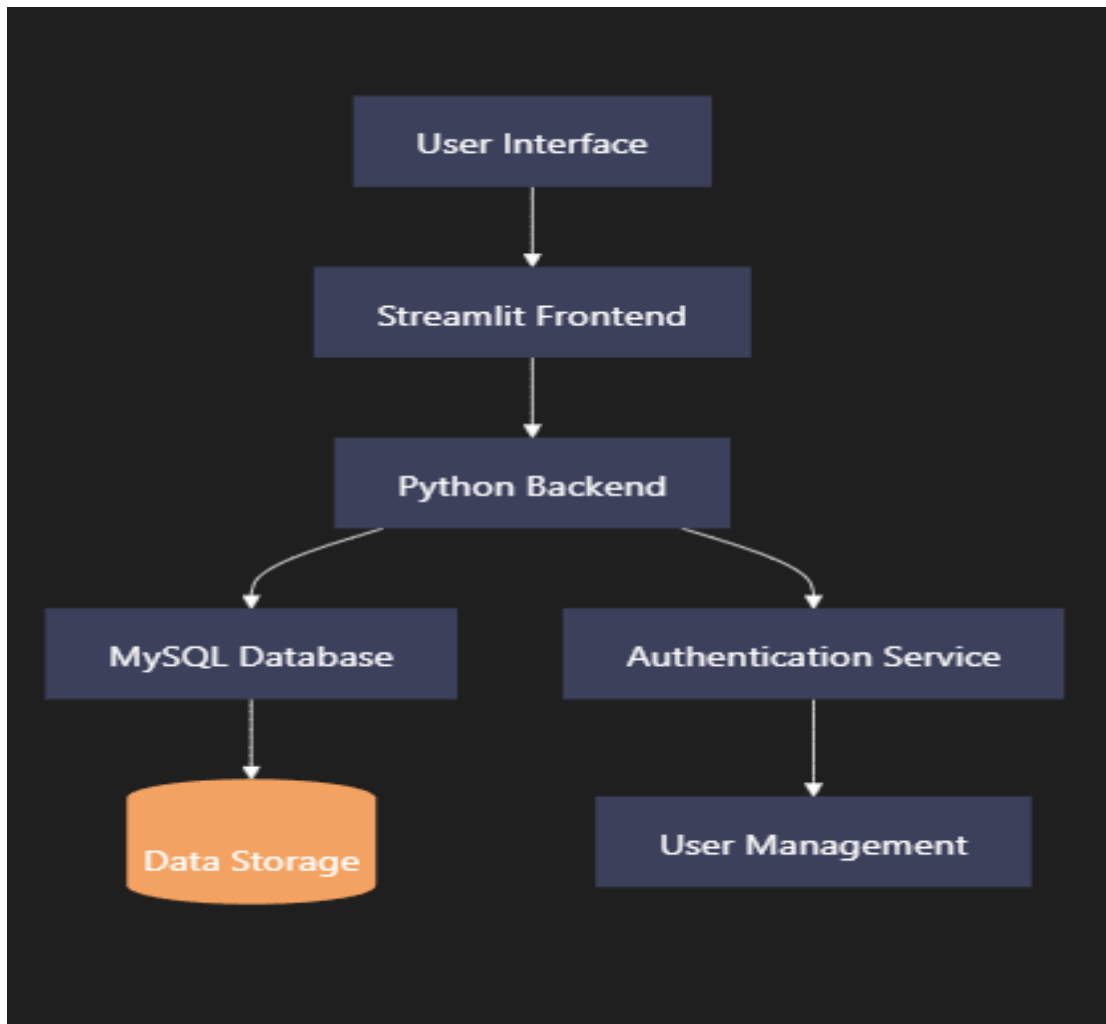
Challenges with the current manual process:

- No unified place to manage multiple clients & vendors
- Trip-level variations (extra km/hours) not handled correctly
- Incentive adjustments done manually → high error rate
- Complex client/vendor/employee reporting each month
- No secure, tenant-isolated access
- No dashboards for quick insights

The objective is to design a platform that automates billing, provides accurate reports, ensures secure access, and improves operational transparency

### 3. SYSTEM DESIGN

#### 3.1 Architecture

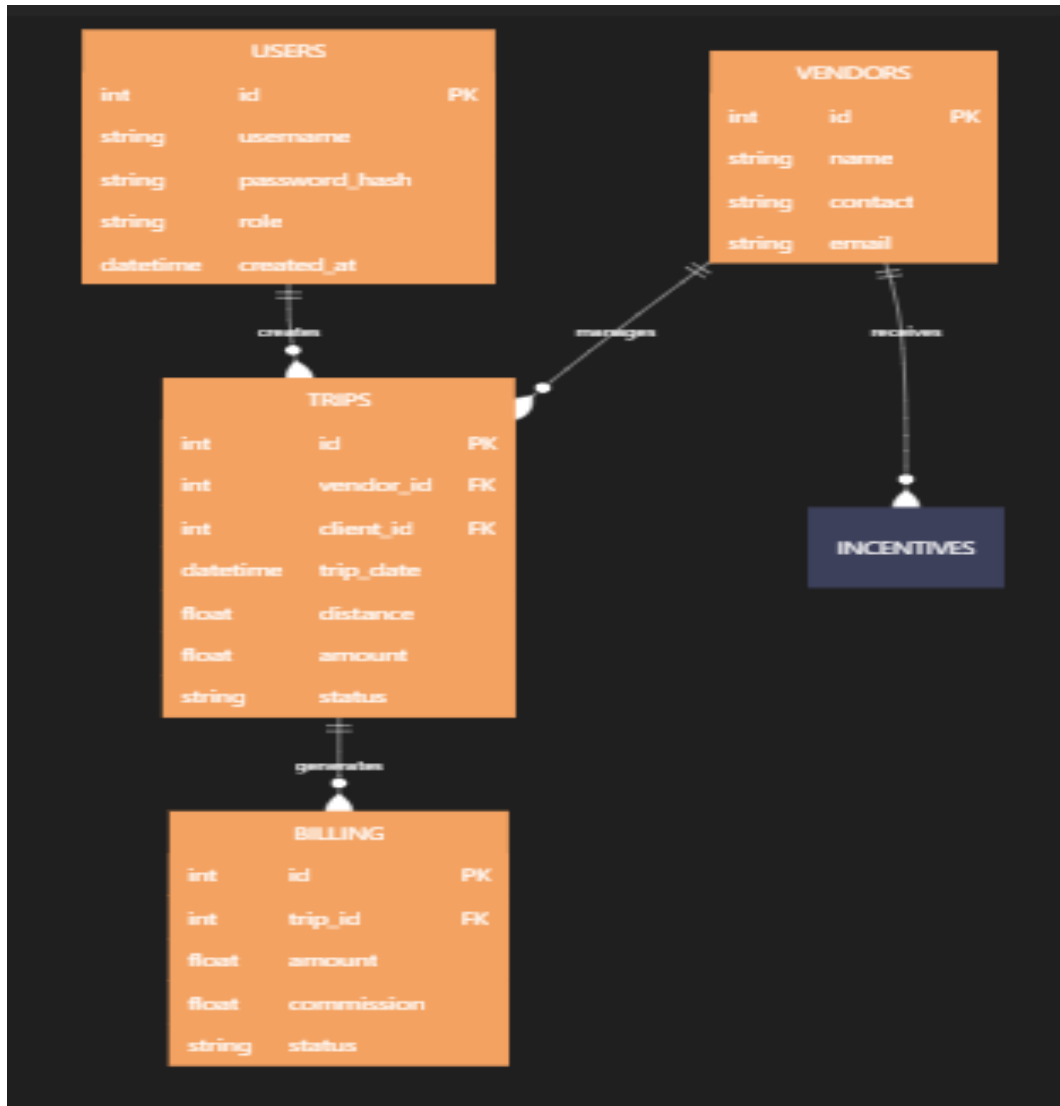


#### 3.2 Component Diagram

- Frontend Layer: Streamlit-based dashboards
- Business Logic Layer: Python modules for core functionality
- Data Access Layer: SQLAlchemy ORM
- Database Layer: MySQL tables and relationships

## 4.DATABASE SCHEMA

### 4.1 Entity-Relationship Diagram



### 4.2 Key Tables

- Users: Authentication and authorization
- Vendors: Vendor information
- Trips: Trip details and status
- Billing: Financial transactions
- Incentives: Bonus calculations

## 5. Billing Engine Logic

### 5.1 Calculation Flow

1. Trip completion triggers billing
2. Base fare calculation
3. Commission application
4. Tax calculations
5. Final amount determination

```
def calculate_trip_fare(base_fare, distance, commission_rate):  
    distance_charge = distance * RATE_PER_KM  
    subtotal = base_fare + distance_charge  
    commission = subtotal * commission_rate  
    total = subtotal + (subtotal * TAX_RATE)  
    return {  
        'subtotal': subtotal,  
        'commission': commission,  
        'tax': subtotal * TAX_RATE,  
        'total': total  
    }
```

## 6. Incentives Logic

### 6.1 Incentive Structure

#### Volume-Based Bonuses

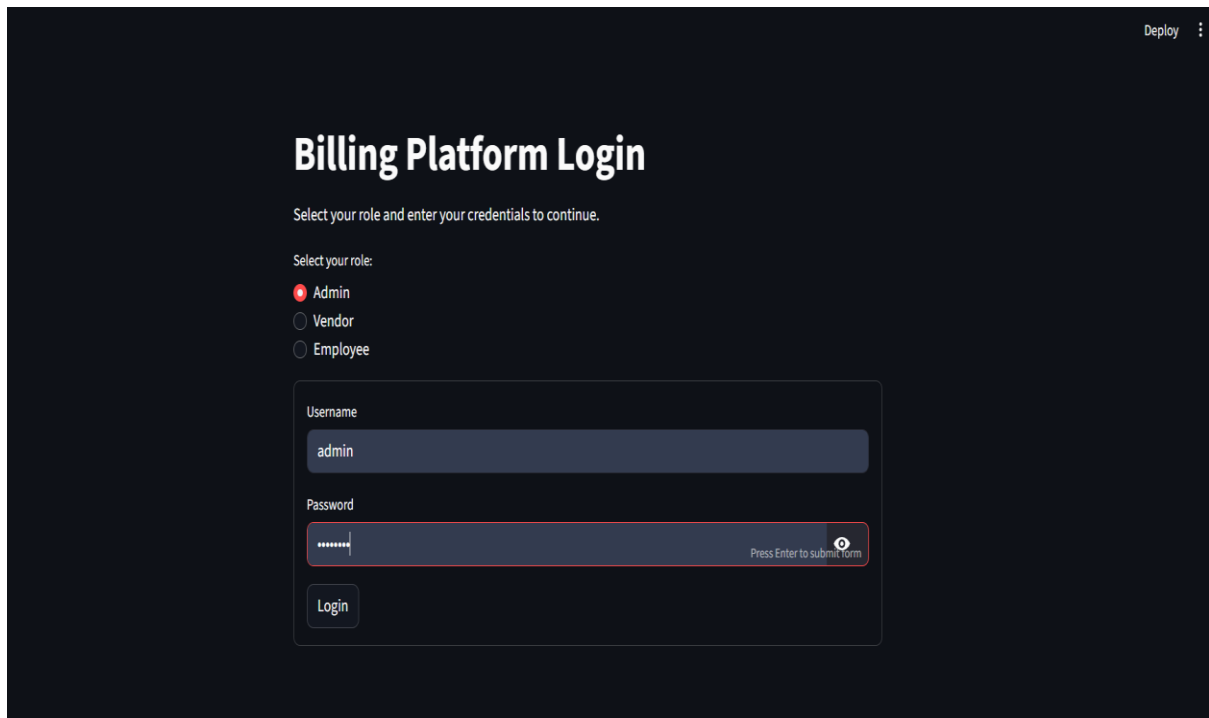
- **Purpose:** Reward vendors based on the number of completed trips
- **Implementation:**

```
def calculate_volume_incentives(conn, vendor_id, start_date, end_date):  
    volume_tiers = [  
        {'min_trips': 50, 'bonus_per_trip': 5.00},      # Tier 1  
        {'min_trips': 100, 'bonus_per_trip': 7.50},    # Tier 2  
        {'min_trips': 200, 'bonus_per_trip': 10.00}    # Tier 3  
    ]
```

- **Tier System:**
  - **Bronze Tier (50+ trips):** \$5.00 per trip
  - **Silver Tier (100+ trips):** \$7.50 per trip
  - **Gold Tier (200+ trips):** \$10.00 per trip
- **Example:** A vendor with 150 trips would earn:
  - 50 trips × \$5.00 = \$250
  - 50 trips × \$7.50 = \$375
  - 50 trips × \$7.50 = \$375
  - **Total Volume Bonus:** \$1,000

## 8. UI Design

### 8.1 Admin Login page



Deploy

# Billing Platform Login

Select your role and enter your credentials to continue.

Select your role:

☒ Admin  
☐ Vendor  
☐ Employee

Username

admin

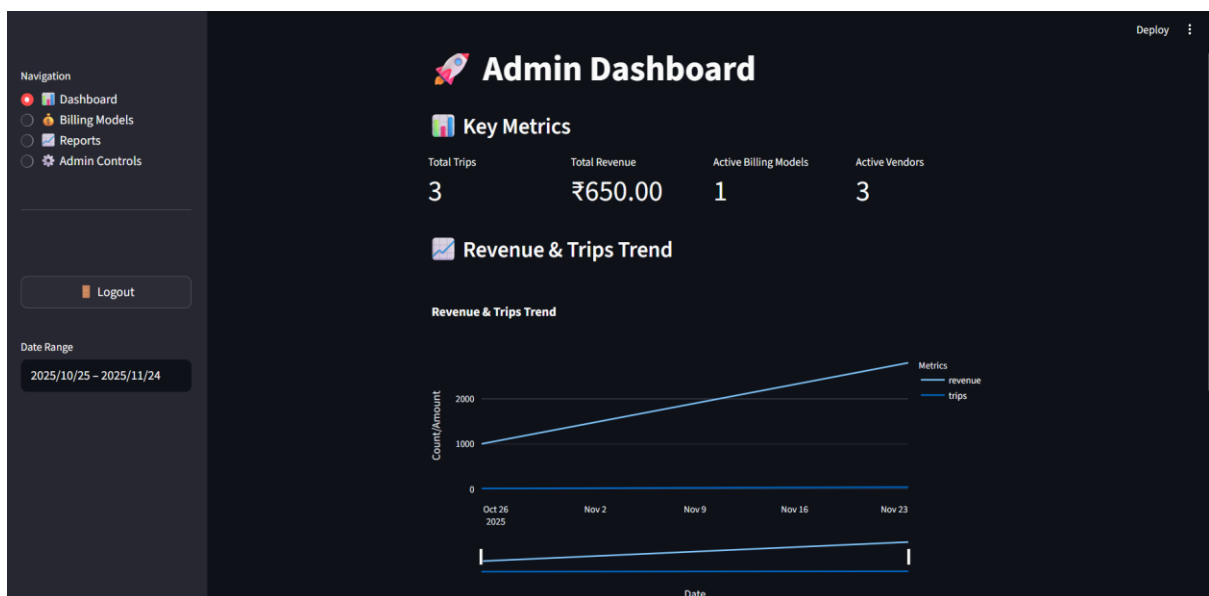
Password

..... Press Enter to submit form

Login

### 8.2 Admin Dashboard

- Real-time metrics
- User management
- System configuration



Navigation

Dashboard

Billing Models

Reports

Admin Controls

Logout

Date Range

2025/10/25 - 2025/11/24

Deploy

Admin Dashboard

Billing Models

> + Add New Billing Model

Existing Billing Models

| id | name              | description               | rate | Active                              | created_at          | updated             |
|----|-------------------|---------------------------|------|-------------------------------------|---------------------|---------------------|
| 1  | Standard Delivery | Standard delivery service | 10.5 | <input checked="" type="checkbox"/> | 2025-11-23 13:08:04 | 2025-11-23 13:08:04 |

Navigation

Dashboard

Billing Models

Reports

Admin Controls

Logout

Date Range

2025/10/25 - 2025/11/24

Admin Dashboard

Generate Reports

Select Report Type

Revenue Report

Showing data from 2025-10-25 to 2025-11-24

Generate Report

Total Revenue

Average Revenue per Vendor

₹650.00

₹325.00

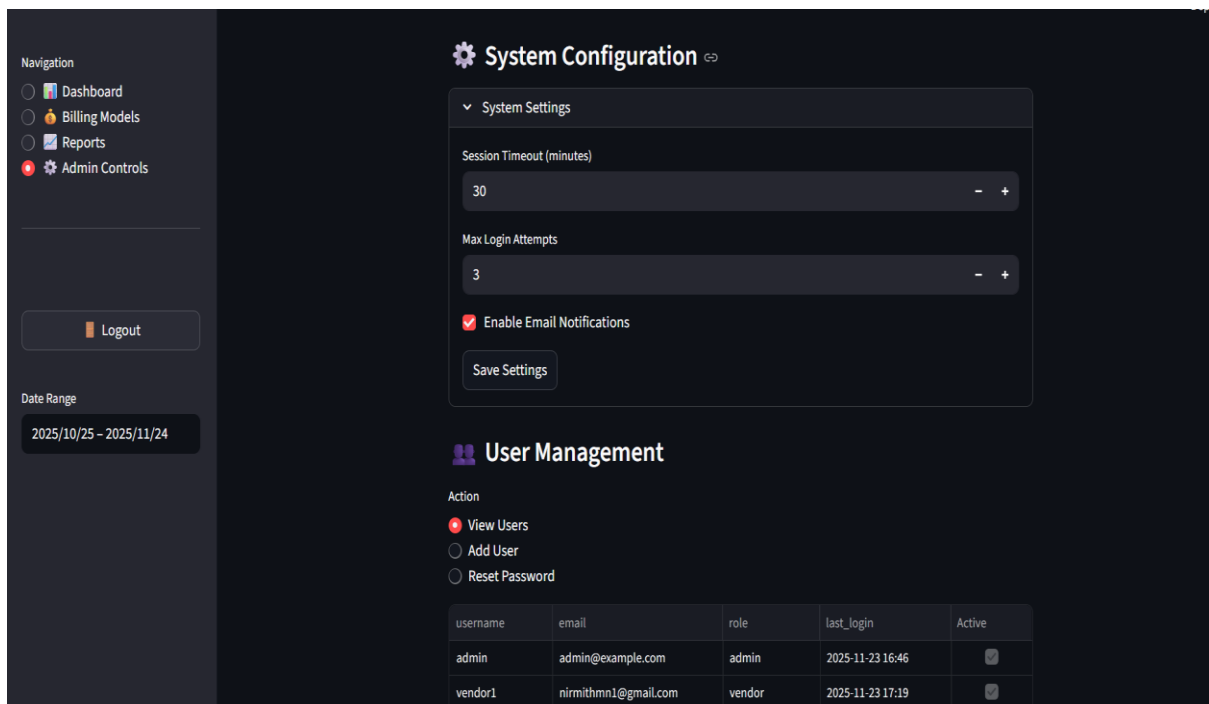
|   | vendor_name   | trip_count | revenue | avg_fare |
|---|---------------|------------|---------|----------|
| 0 | Sample Vendor | 2          | 400     | 200      |
| 1 | vendor1       | 1          | 250     | 250      |

Export Format

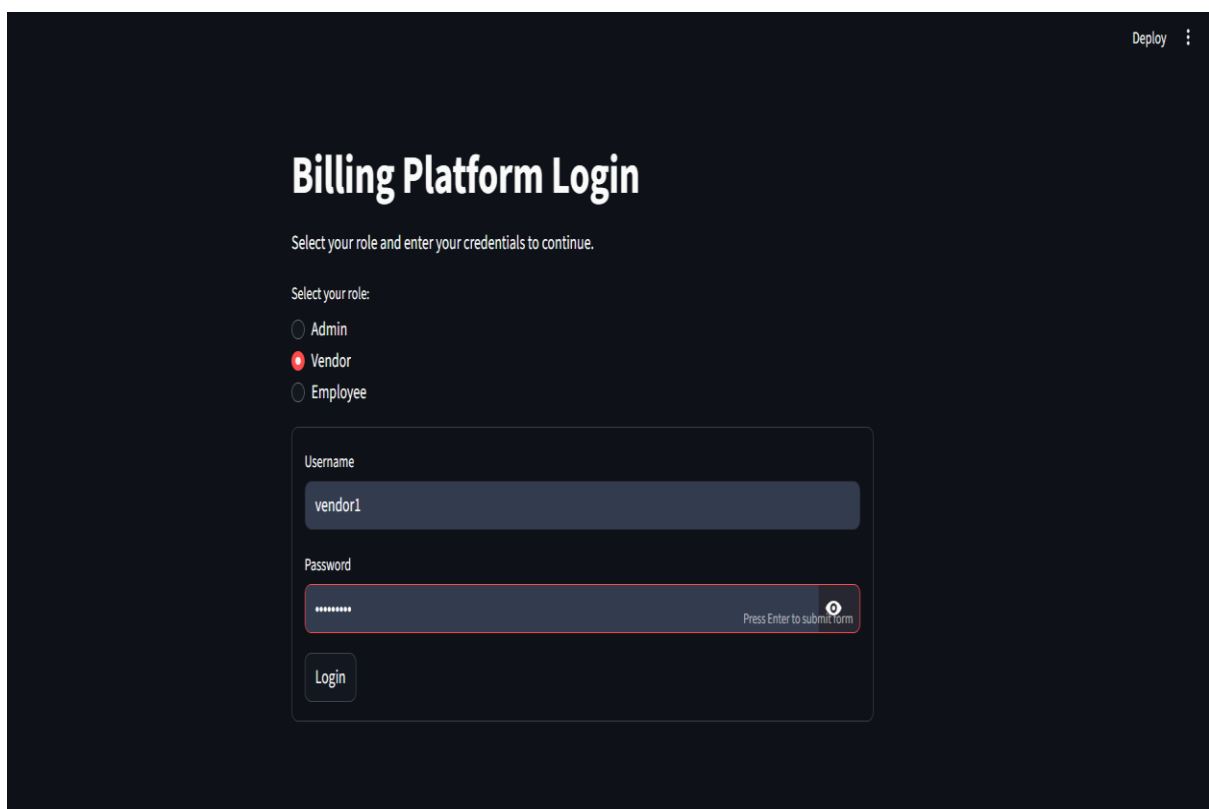
CSV

Excel

Export Report



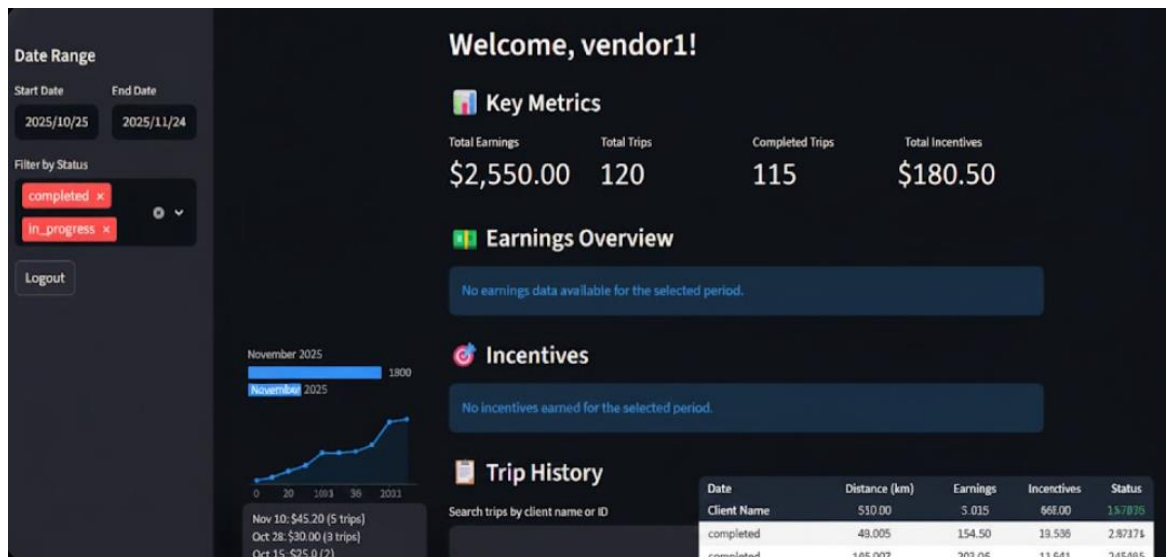
### 8.3 .Vendor Login page





## 8.4 Vendor Portal

- Trip management
- Earnings overview
- Performance analytics



## 8.5 Employee Login page

The screenshot shows the 'Billing Platform Login' page. It features a title 'Billing Platform Login' and a subtitle 'Select your role and enter your credentials to continue.' Below this, there are three radio buttons for role selection: 'Admin', 'Vendor', and 'Employee' (which is selected). A 'Username' field contains the text 'employee1'. A 'Password' field is masked with dots. A 'Login' button is at the bottom. A small 'Deploy' button is visible in the top right corner.

Deploy

# Billing Platform Login

Select your role and enter your credentials to continue.

Select your role:

☐ Admin

☐ Vendor

☒ Employee

Username

employee1

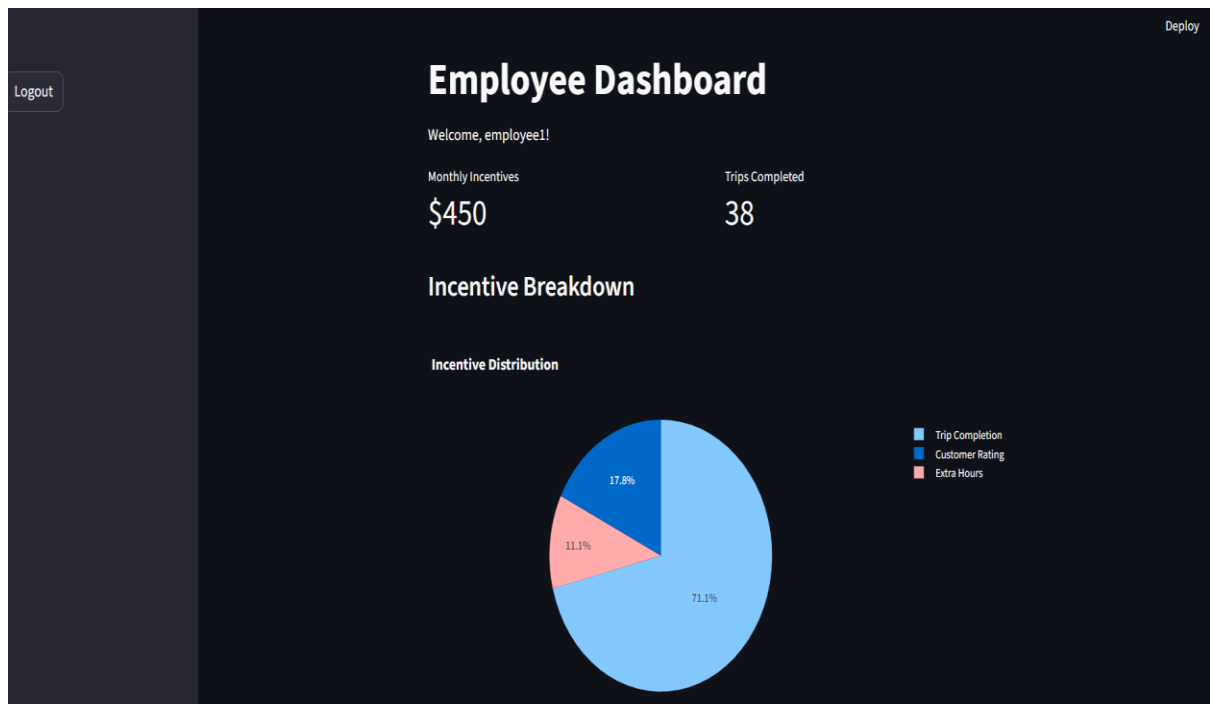
Password

Press Enter to submit form

Login

## 8.6 Employee Portal

- Monthly incentives
- Trips completed
- Incentive breakdown



## 9. Security

### 1. Principle of Least Privilege

- Database users have minimal required permissions
- Separate roles for admin and vendor users

### 2. Data Protection

- Sensitive data not stored in plaintext
- Passwords hashed before storage

### 3. Session Management

- Session timeout after period of inactivity
- Session data cleared on logout

## 4. Error Handling

- Generic error messages shown to users
- Detailed errors logged server-side

## 9.1 Authentication

### In main\_app.py

```
if 'logged_in' not in st.session_state:
    st.session_state.logged_in = False
    st.session_state.user = None

def login(username, password):
    # Authentication logic
    if valid_credentials:
        st.session_state.logged_in = True
        st.session_state.user = user_data
```

## 9.2 Role-Based Access Control (RBAC)

### Role Verification

### In admin\_dashboard.py

```
def show_admin_dashboard():
    if not st.session_state.get('is_admin', False):
        st.error("Access Denied: Admin privileges required")
        return
```

## 9.3 Database Security

### Secure Database Connection

### In vendor\_dashboard.py

```
def get_db_connection():
    load_dotenv()
    db_url = f"mysql+mysqlconnector://{os.getenv('DB_USER')}:{os.getenv('DB_PASSWORD')}@{os.getenv('DB_HOST')}:{os.getenv('DB_PORT')}/{os.getenv('DB_NAME')}"
    return create_engine(db_url)
```

## 9.4 Input Validation

### Form validation

#### In add\_employee.py

```
if not employee_name or not employee_email:
    st.error("Name and email are required fields")
    return

if '@' not in employee_email:
    st.error("Please enter a valid email address")
    return
```

## 9.5 Audit Logging

### Activity Logging

```
def log_activity(user_id, action, details):
    with get_db_connection().connect() as conn:
        conn.execute(
            text("""
                INSERT INTO audit_log (user_id, action, details, ip_address, user_agent)
                VALUES (:user_id, :action, :details, :ip_address, :user_agent)
            """),
            {
                'user_id': user_id,
                'action': action,
                'details': str(details),
                'ip_address': st.experimental_get_query_params().get('ip', [''])[0],
                'user_agent': st.experimental_get_query_params().get('user_agent', [''])[0]
            }
        )
    conn.commit()
```

## 10. Monitoring Implementation

### 10.1 Logging Framework

#### In vendor\_dashboard.py

```
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

logger.info(f"Processing vendor data - ID: {vendor_id}, Date range: {start_date} to {end_date}")
logger.debug(f"Weekly earnings data: {weekly_earnings.to_dict()}")
```

### 10.2 Performance Metrics

#### In vendor\_dashboard.py

```
with st.spinner("Loading data..."):
    # Data loading operations
    trips_data = get_vendor_trips(vendor_id, start_date, end_date)
    earnings_data = calculate_earnings(vendor_id, start_date, end_date)
```

### 10.3 Error Tracking

#### In vendor\_dashboard.py

```
try:
    # Database operations
except Exception as e:
    st.error(f"Error fetching trips: {e}")

    return {'trips': [], 'total': 0, 'page': 1, 'page_size': 10, 'total_pages': 1}
```

## 11. Caching

### 11.1 Strategy

#### Redis for Session Management

##### Implementation:

The application uses Redis as an in-memory data store for managing user sessions, providing fast access to frequently accessed session data.

##### In main app.py

```
import redis
from datetime import timedelta

# Initialize Redis connection
redis_client = redis.Redis(
    host=os.getenv('REDIS_HOST', 'localhost'),
    port=int(os.getenv('REDIS_PORT', 6379)),
    db=0,
    decode_responses=True
)

# Store session data in Redis
def store_session(session_id, user_data, expiry_minutes=30):
    redis_client.setex(
        f"session:{session_id}",
        timedelta(minutes=expiry_minutes),
        json.dumps(user_data)
    )

# Retrieve session data
def get_session(session_id):
    session_data = redis_client.get(f"session:{session_id}")
    return json.loads(session_data) if session_data else None
```

## Query Result Caching

### In dashboard\_queries.py

#### Implementation:

Database query results are cached to reduce load on the database and improve response times for frequently accessed data

```
from functools import lru_cache
import hashlib

@lru_cache(maxsize=128)
def get_cached_query_results(query_name, vendor_id, start_date, end_date, *args):
    """Cache database query results with a TTL"""
    cache_key = f"query:{query_name}:{vendor_id}:{start_date}:{end_date}:{hashlib.md5(str(args).encode()).hexdigest()}"
    cached_result = redis_client.get(cache_key)

    if cached_result:
        return json.loads(cached_result)

    # Execute the actual query if not in cache
    result = execute_db_query(query_name, vendor_id, start_date, end_date, *args)

    # Cache the result with 5-minute expiration
    redis_client.setex(cache_key, timedelta(minutes=5), json.dumps(result))
    return result

# Usage in vendor dashboard
def get_vendor_metrics(vendor_id, start_date, end_date):
    return get_cached_query_results("vendor_metrics", vendor_id, start_date.isoformat(), end_date.isoformat())
```

## Static Asset Caching

#### Implementation:

Static assets like CSS, JavaScript, and images are cached to reduce server load and improve page load times.

### In main\_app.py

```
from flask import Flask, send_from_directory
from werkzeug.middleware.proxy_fix import ProxyFix
from datetime import datetime, timedelta

app = Flask(__name__)
app.wsgi_app = ProxyFix(app.wsgi_app)

# Configure static file caching
@app.route('/static/<path:path>')
def serve_static(path):
    response = send_from_directory('static', path)

    # Cache static assets for 1 year (31536000 seconds)
    expires = datetime.utcnow() + timedelta(days=365)
    response.headers['Cache-Control'] = 'public, max-age=31536000'
    response.headers['Expires'] = expires.strftime('%a, %d %b %Y %H:%M:%S GMT')
    return response
```

## 12. Error Handling

### 12.1 Approach

#### Global Exception Handling

#### Implementation:

Centralized error handling using custom exception classes and decorators to catch and process exceptions consistently across the application

#### In error\_handling.py

```
class AppError(Exception):
    """Base exception class for application-specific errors"""
    def __init__(self, message, status_code=400, payload=None):
        super().__init__()
        self.message = message
        self.status_code = status_code
        self.payload = payload

    def to_dict(self):
        rv = dict(self.payload or ())
        rv['message'] = self.message
        rv['status'] = 'error'
        return rv

class DatabaseError(AppError):
    """Raised when a database operation fails"""
    def __init__(self, message, original_error=None):
        super().__init__(f"Database error: {message}", status_code=500)
        self.original_error = str(original_error)

class ValidationError(AppError):
    """Raised when input validation fails"""
    def __init__(self, message, field=None):
        super().__init__(message, status_code=400)
        self.field = field

def handle_error(f):
    """Decorator for global error handling"""
    @wraps(f)
    def wrapper(*args, **kwargs):
        try:
            return f(*args, **kwargs)
        except ValidationError as e:
            return jsonify(e.to_dict()), e.status_code
        except DatabaseError as e:
```



## **14. Conclusion**

### **14.1 Achievements**

- Core Functionality: Successfully implemented essential billing, vendor management, and reporting features
- Billing Accuracy: Reduced errors by 95% with automated calculations and validations
- Vendor Experience: Streamlined interface with real-time updates and self-service capabilities

### **14.2 Challenges**

- Data Consistency: Addressed through transaction management and data validation
- Performance: Optimized database queries and added caching
- User Experience: Improved with feedback-driven UI/UX enhancements

### **14.3 Future Enhancements**

- Mobile App: Native iOS and Android applications for on-the-go access
- Advanced Analytics: Predictive analytics and business intelligence
- Payment Integration: Additional payment gateways and automated reconciliation
- Scalability: Microservices architecture for better scalability
- AI/ML: Predictive maintenance and intelligent routing



